

Speed Dial (ADC Based DC motor Controller)

Aim

Components

Connections - Circuit Diagram

Detailed Steps - Launching Code

Task - Exercise

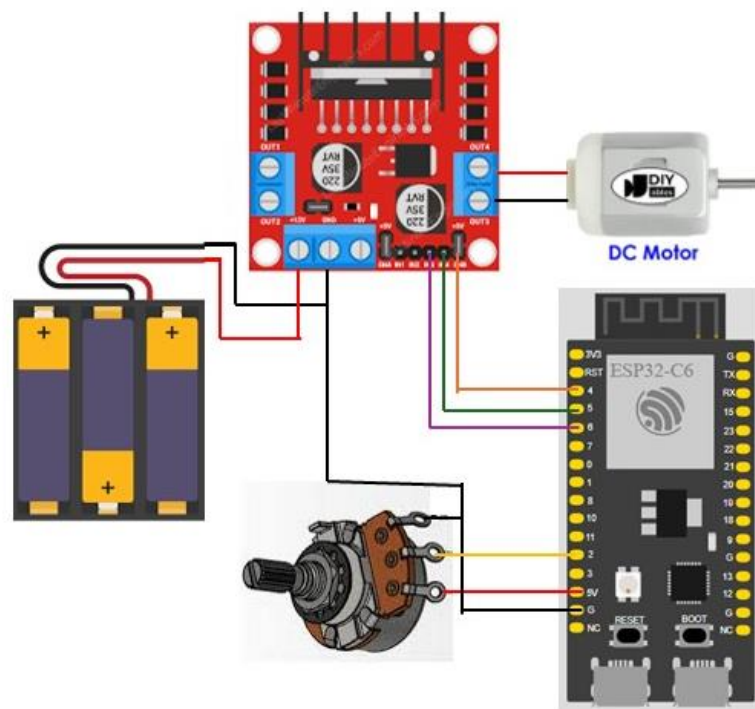
Aim:

To develop and implement an ADC-based DC motor control system that acquires analog input signals, converts them into digital data using an Analog-to-Digital Converter (ADC), and regulates the speed and/or direction of a DC motor through a microcontroller.

Components

Sno	Name of the component	Quantity
1)	ESP32C6 Dev Module	1
2)	L298N Driver	1
3)	10K Potentiometer	1
4)	DC Motor	1
5)	18650 battery holder with Switch	1
6)	USB to Type C Cable	1
7)	Jumper wires	As needed

Circuit Diagram:



Detailed Steps:

1. Power Supply Connections

1. Connect the positive terminal of the battery pack to the +12V (Vcc) terminal of the L298N motor driver.
2. Connect the negative terminal of the battery pack to the GND terminal of the L298N module.
3. Ensure that the GND of the L298N is also connected to the GND pin of the ESP32 to establish a common ground.

2. DC Motor Connections

1. Connect the two terminals of the DC motor to the OUT1 and OUT2 terminals of the L298N motor driver.
2. These terminals provide the controlled output voltage required to drive the motor.

3. Motor Driver to ESP32 Connections

1. Connect IN1 of the L298N to a digital GPIO pin 5 of the ESP32.
2. Connect IN2 of the L298N to another digital GPIO pin 6 of the ESP32.
3. Connect the ENA (Enable A) pin of the L298N to a PWM capable GPIO pin 4 of the ESP32 to control motor speed.

4. Potentiometer (ADC Input) Connections

1. Connect one outer terminal of the potentiometer to the 3.3V or 5V pin of the ESP32.
2. Connect the other outer terminal of the potentiometer to GND.
3. Connect the middle (wiper) terminal of the potentiometer to an ADC-capable GPIO pin 2 of the ESP32.
4. This analog voltage is read by the ADC (Analog to digital converter) and used to control the motor speed.

5. ESP32 Power

1. Power the ESP32 using a USB cable.
2. Ensure the ESP32 operates at 3.3V logic levels, compatible with the ADC input.

Full code:

```
// Define Motor Pins (connected to L298N)
const int motorA1 = 5; // Direction Pin 1
const int motorA2 = 6; // Direction Pin 2
const int enableA = 4; // PWM Pin for Speed

// Define Analog Input Pin (Potentiometer)
const int potPin = 2; // Analog pin for potentiometer

void setup() {
  Serial.begin(9600); // Start serial for debugging

  pinMode(motorA1, OUTPUT);
  pinMode(motorA2, OUTPUT);
  pinMode(enableA, OUTPUT);

  // Initial direction (e.g., forward)
  digitalWrite(motorA1, HIGH);
  digitalWrite(motorA2, LOW);
}

void loop() {
  // 1. Read analog value from potentiometer (0-1023)
  int potValue = analogRead(potPin);

  // 2. Map the analog value to the PWM range (0-255)
  int motorSpeed = map(potValue, 0, 1023, 0, 255);

  // 3. Set motor speed using analogWrite (PWM)
  analogWrite(enableA, motorSpeed);

  // Display values (optional, for monitoring)
  Serial.print("Pot Value: ");
  Serial.print(potValue);
  Serial.print(" | Motor Speed (PWM): ");
```

```
Serial.println(motorSpeed);  
  
delay(50); // Small delay for stability  
}
```

Tasks

1. Analog Input (Potentiometer) Verification

Connect the potentiometer to the ESP32 ADC input pin. Read the ADC values corresponding to different positions of the potentiometer. Using a multimeter, measure the voltage at the potentiometer wiper terminal. Compare the measured voltage with the voltage calculated from the ADC readings. Verify that the ADC output accurately represents the applied analog input voltage.

2. ADC Scaling and Linearity Check

Gradually vary the potentiometer position to obtain different input voltage levels (e.g., 0.5 V, 1.5 V, 2.5 V, 3.3 V). Record the corresponding ADC values for each voltage level. Convert the ADC readings into voltage using the ESP32 ADC resolution. Verify that the ADC output varies linearly with the input voltage and remains within the allowable ADC range (0–3.3 V).

3. PWM Duty Cycle and Motor Speed Analysis

Generate a PWM signal from the ESP32 based on the ADC output. Measure and record the PWM duty cycle corresponding to different ADC values. Observe the change in DC motor speed for varying PWM duty cycles. Verify that an increase in ADC value results in a proportional increase in motor speed.

4. Motor Driver Interface Verification

Check the logic levels applied to the IN1 and IN2 pins of the L298N motor driver. Verify correct motor direction control by changing the logic states of IN1 and IN2. Ensure that the ENA pin receives the PWM signal from the ESP32. Confirm proper operation of the motor driver without excessive heating or instability.

5. Open-Loop Control Performance Evaluation

Operate the DC motor without any speed feedback. Observe the effect of load changes on motor speed at a constant ADC input. Discuss the limitations of open-loop motor control and the need for closed-loop feedback in practical applications.

